

# One Agent, Anywhere: Personal Agents Accompanying Users Across Applications

Zhenyu Zhao<sup>1</sup>, Roy Zhao<sup>2</sup>, Yuhang An<sup>1</sup>, Zhide Yin<sup>1</sup>

<sup>1</sup> Independent Researcher

<sup>2</sup> Paul G. Allen School of Computer Science & Engineering, University of Washington

Contact: Roy Zhao, royzh@cs.washington.edu

Position paper - September 6, 2026

## Abstract

Applications today integrate their own AI assistants. We explore a different future: users bring the same persistent personal agent with them across applications. Each application contributes local state and domain intelligence; the personal agent contributes the user's continuing context, preferences, and goals. We argue that independently operated applications should support one continuing user-agent collaboration while retaining authority over their services. The user governs participation and delegation. Message delivery and bidirectional context exchange connect this persistent relationship to local application work. A reference architecture supports direct application connections and broker-assisted awareness of user visits without a public inbound agent endpoint. Through a proposed Ruth shopping scenario spanning Telegram and two stores, we examine authority, privacy, adoption, and how to test the model. The contribution is an architectural position and its collaboration semantics; the mechanisms reuse established infrastructure. We report no completed cross-application implementation or empirical results.

## 1. Introduction and position

A user's decision can span several applications, each contributing different facts and operations. The goal persists across these transitions, while the information needed to make progress changes: a selected product, a delivery estimate, or a comparison already discussed. Application integration must connect that local work to the user's continuing discussion and priorities.

Two discontinuities motivate our position. **Conversation discontinuity** arises when assistance is organized around separate app-local histories: earlier decisions do not automatically accompany the user. **Context discontinuity** remains possible even with a persistent agent: remembering the discussion does not reveal what the user currently sees or selects inside an app. Shared memory alone leaves the second gap; access to app state alone leaves the first. Applications need to support both forms of continuity while retaining their own access and business boundaries.

Consider a user discussing comfortable shoes and a budget with their personal agent on Telegram. They open two stores, asking "What about this pair?" in the first and "Is this better than the first?" in the second. The current selection comes from the second store; the first candidate and the user's priorities come from the earlier discussion. The user should be able to continue that comparison inside either store:

**An application boundary should not end the user's ongoing collaboration with their personal agent.**

We call this a *personal-agent-centric* collaboration model. Independently operated applications join a continuing relationship, contributing local context and domain intelligence. Users browse and decide inside the apps, with their agent participating in the conversation and taking delegated actions when requested. The architectural requirement is continuity of collaboration across these changing settings.

Persistent-agent architectures address continuity across execution and interaction bindings [1]; mixed-initiative systems study human action alongside automated assistance [2]. We build on these foundations to specify how applications can participate in the ongoing relationship and what evidence would support or challenge this position.

## 2. A collaboration model with complementary intelligence

**The personal agent maintains continuity; applications retain domain authority; the user governs their collaboration.** An application can contribute its own agent, language model, or domain logic within this division of responsibility.

| Participant    | Contribution                                                                    | Authority                                                   |
|----------------|---------------------------------------------------------------------------------|-------------------------------------------------------------|
| User           | Goals, judgments, corrections, and decisions                                    | Chooses assistance and authorizes participation and actions |
| Personal agent | Continuing conversation, memory, preferences, and reasoning across applications | Operates within the user's delegated authority              |
| Application    | Local state, domain intelligence, and available operations                      | Controls its services, business rules, and permitted access |

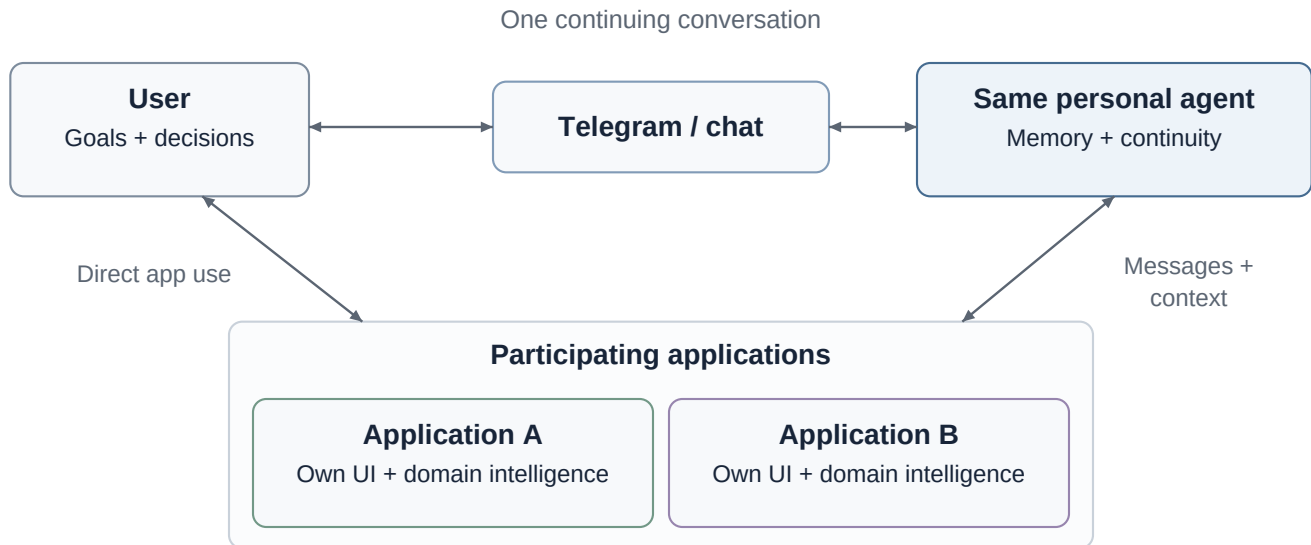


Figure 1. Three-party collaboration. The user works directly in applications and continues with the same agent through their interfaces or Telegram. Applications retain domain authority while exchanging messages and context with the agent. Arrows show interaction and data flow; the agent initiates connections to app APIs.

**Continuity belongs to the relationship.** An app visit provides a setting for an existing conversation. The agent preserves links between earlier choices and current questions; an app session identifies a message's source and a reply destination. Continuity requires the relevant agent state and history to persist, beyond a shared name or persona. Reasoning with that history need not expose it to every app.

**User control must be operational.** Users should be able to choose their agent, retain its continuing state independently of an app, and authorize or revoke connections. These are the properties we mean by user-governed. Hosting may be local or remote, without ownership of model weights; exportability, migration, and revocation require implementation support.

**Reasoning remains complementary.** A store evaluates availability and delivery options using its domain logic. The personal agent compares those conclusions with the user's budget and earlier decisions. An authorized budget disclosure can also inform the store's ranking. Each side can tune its own reasoning, with conclusions attributed to their source when they disagree.

**Participation is bounded.** An app supplies relevant context and receives authorized messages and selected personal context. Access to one turn grants no access to all memory; knowledge of a selected product grants no purchasing authority. These boundaries remain part of the collaboration even as the agent carries context across applications.

### 3. Reference architecture and application presence

Applications host authenticated interfaces; the personal agent connects as a client. A messaging adapter offers another entry point into the conversation. Apps can use their own UI or reusable components across web, mobile, and messaging surfaces; an SDK could package the two contracts and presence reporting. Direct app connections and an optional notification broker support different scales of participation while keeping the agent in an outbound client role.

#### 3.1 Two collaboration contracts

| Contract         | Application to personal agent                                  | Personal agent to application                                  |
|------------------|----------------------------------------------------------------|----------------------------------------------------------------|
| Message delivery | User message, source session, and turn identity                | Reply, original-message reference, and delivery destination    |
| Context exchange | Current selection, relevant page state, and domain conclusions | Authorized preferences, constraints, and collaboration outputs |

The contracts describe collaboration semantics: messages from different apps become turns in one continuing conversation, and authorized context flows between its persistent state and local application work. They can share a transport and use existing interaction protocols and libraries [3, 4]. No new wire standard is required.

A message must remain associated with the context in which it was sent. If the user changes products while an answer is being generated, "this pair" still refers to the earlier selection. Source application, session, object identifier, and context revision make that association explicit. Stable message and output identifiers support deduplication and recovery without turning a retransmission into a new conversation turn.

Ordinary application APIs remain available for catalog queries and authorized orders. Their transaction rules and permissions remain with the app; actions can contribute results to the conversation without becoming another collaboration contract.

#### 3.2 Task-scoped application connections

For a bounded task, the agent selects a few authorized apps and reads their activity, messages, and context through app-hosted event feeds. Server-Sent Events (SSE) [10] or cursor-based polling can carry these reports without invoking model reasoning for transport maintenance. Completion or cancellation closes the connections. Automatic awareness covers only the connected apps.

A smaller demo can omit activity reporting and reply to each message's source session. A message from another app continues the conversation there without requiring an estimate of the user's current app.

#### 3.3 A shared notification queue

For visits across many apps, a broker hosts a notification queue for the user's personal agent. Authorized account linking gives an app a queue destination and scoped publishing credential. Its SDK reports visits and foreground activity to its backend, which publishes events to the queue. One logical outbound subscription lets the agent learn of these visits without a persistent connection to every app.

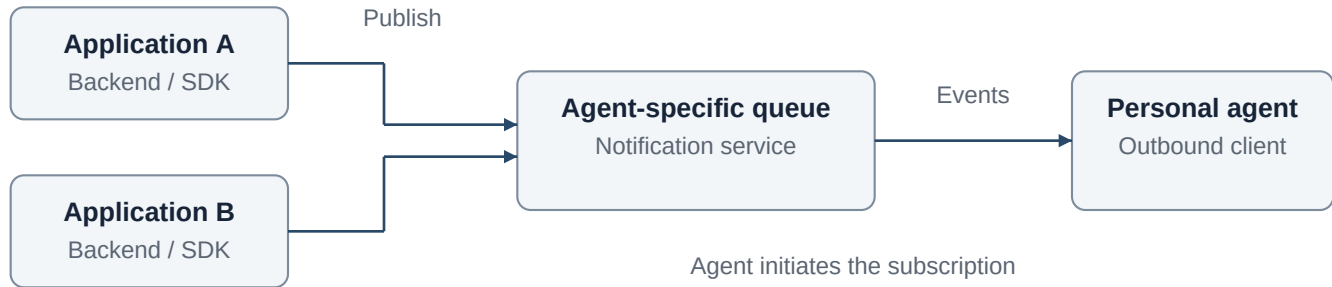


Figure 2. Optional presence infrastructure. Arrows show notification flow. The agent initiates the queue subscription, then calls the relevant app's existing message/context interfaces. The queue need not carry conversation text or personal memory.

An event identifies the app relationship, session, device, activity, and freshness. CloudEvents offers reusable identity and source fields [11]. After a presence or message-available notification, the agent reads the app's context and messages and submits replies through the same two contracts. The broker exposes the publishing API; the agent needs no public inbound endpoint. Independent hosting or an SDK provider could supply the service.

The queue subscription may persist across tasks under user authorization, with app connections opened as needed. Coverage extends to authorized, integrated apps, including later returns. Event traffic still grows with activity. Scoped publishing grants isolate app identities, prevent reading other apps' events, and permit revocation of future publishing. Broker availability and activity-metadata protection become additional responsibilities.

Presence is evidence of recent interaction. Reports can be delayed or conflict across devices; expiration, deduplication, and cursor or snapshot recovery address stale and repeated events. An explicit message identifies a reply destination. Uncertain presence should not silently relocate a pending reply, and a visit grants no new access to memory or actions.

#### 4. Ruth: a proposed shopping scenario

Ruth is an Enoch-derived project built on a persistent-agent foundation [1]. The proposed cross-application demo uses two preauthorized stores and source-session replies, initially omitting automatic presence reporting and the broker. These integrations remain unimplemented.

1. **Begin in Telegram.** The user sends /shop with shoe preferences and a budget. Ruth loads a registry of two authorized stores and starts reading their message feeds. A Telegram adapter uses native messaging APIs [9].
2. **Recommend concrete candidates.** Ruth calls each app's queryProduct and, where needed, getProduct. It sends recommendations with product links to Telegram. Product references include both the app and product identifiers.
3. **Collaborate in App A.** The user opens a candidate and asks, "What about this pair?" App A queues that message with the selected product and relevant page state. Ruth combines it with the earlier discussion and posts its answer to App A's session. With authorization, it can also share the budget for App A's own filtering.
4. **Continue in App B.** The user opens another candidate and asks, "Better than the first?" Ruth resolves the current selection using App B's context and the earlier candidate using the continuing conversation. App B renders the answer in its own chat interface. Returning to App A uses the same process.
5. **Finish with an action.** Once the user requests a specific purchase, Ruth retrieves current product details and invokes the selected app's orderProduct under that authorization. A simulated order result is delivered to the source app and Telegram, after which app polling stops.

Different storefronts make application changes visible; budget, candidate references, and discussion persist across turns. Completing this scenario would establish a controlled example. Independent integration and comparative evaluation are needed to assess generality and benefit.

## 5. Relationship to existing work

**Persistent agents and mixed initiative.** RIPA separates persistent identity, memory, and software body from replaceable execution and interaction bindings [1]. Code Is the Body studies governed software evolution and descent [8]. We examine how such an agent participates in application workflows; switching apps need not migrate its process. Horvitz's mixed-initiative principles [2] motivate combining direct application use with continuing personal-agent assistance.

**Interfaces and personal-agent continuity.** AG-UI standardizes agent-application events [3]; CopilotKit connects interfaces to configurable agents and supplies app context [4]. These mechanisms can support our design. Our architectural focus is continuity across independently operated applications: different interfaces connect to the same user-governed persistent personal agent, preserving one conversation and memory.

**Delegation and continuing collaboration.** AgentPort's public engineering note connects user-owned agents to app capabilities through scoped grants and encrypted sessions [5]. We read its capability-lending design as emphasizing delegated execution. We emphasize users browsing, comparing, and deciding with the same personal agent across apps that retain their own reasoning. Our model includes delegated actions; AgentPort also describes conversation and memory continuity. The distinction concerns design emphasis and collaboration requirements, not mutually exclusive capabilities.

**Computer use and native collaboration.** Screenshot-driven computer use infers application state through human-facing interfaces and acts through mouse and keyboard controls [12]. We use *AI-native collaboration* to mean that applications explicitly support personal agents as participants through message delivery and bidirectional context exchange, reducing reliance on reconstructing application semantics from a rendered UI. Our design requires app integration. Computer use can also support persistent conversation and human-agent collaboration, and can provide fallback access to apps without these contracts. Claude's documentation identifies visual-grounding and tool-selection errors, latency, and screenshot-related token costs [12]. These motivate testing explicit context exchange; they do not establish superior reliability or lower total cost for our proposal.

**Personal context and coherent tools.** PSI connects generated instruments through a personal-context bus, reporting an autobiographical deployment [6]. The Human Context Protocol report advocates portable, persistent, user-governed memory [7]. We focus on independently intelligent applications joining the personal agent through bounded exchanges. Evaluation should include approaches inspired by this work as alternatives to fragmented app chat.

## 6. Objections and a research agenda

### 6.1 What do explicit collaboration interfaces add?

An application-specific assistant may have better access to internal state, lower latency, and more predictable domain behavior. Cross-application continuity may offer little benefit for a short, self-contained task. The proposed model should therefore be evaluated where decisions depend on information established across applications, while preserving strong application reasoning in the comparison.

Compare app assistants given imported user preferences, a personal agent with a shared transcript and ordinary product tools, and the same agent with message-bound application context. A further condition adds authorized context returned to app-owned filtering or ranking. Hold models, catalogs, tool access, and permissions constant where possible. Measure reference resolution, retention of task constraints, unnecessary clarification, and correctness of recommendations and requested actions. Improvements are hypotheses, not results reported here.

A central agent chat supplied with equivalent application context is a necessary strong baseline. If it performs equally well, the evidence would support context integration without demonstrating an advantage from continuing the conversation inside each app. That outcome would narrow the claim and shift the remaining evaluation toward integration cost and the practical value of in-app collaboration.

Compare screenshot-based context acquisition with app-provided context for the same model, conversation history, user interaction, catalogs, and permissions; hold action APIs constant to isolate context exchange. Measure task success, reference errors, latency, and model calls. Count input/output tokens across personal-agent and app-side models, including screenshots and retries. Report cost per successful task including failed attempts, and integration effort separately. This tests whether reduced UI interpretation yields a net benefit rather than shifting work to the app.

## **6.2 Does persistent personal context create a larger privacy risk?**

Concentrating context in one agent creates a valuable point of failure. User control over that agent does not itself guarantee secure storage, correct disclosure, or trustworthy model behavior. An app also receives information through natural-language answers rendered in its interface, not only through structured context fields.

Evaluate authorized sharing and unauthorized disclosure separately. Test whether app-supplied content can cause the agent to reveal unrelated memories or perform actions beyond the user's request. Preserve provenance and freshness, but do not treat provenance as proof of truth or permission. Revocation should stop future access; it cannot make a recipient forget plaintext already received. The relevant question is whether useful continuity can be obtained with a small and understandable disclosure boundary.

## **6.3 Will applications participate, and can the contract remain small?**

Applications need reasons to integrate a personal agent: potentially better task completion, less repeated preference collection, or access to users who already rely on an agent. These are possible incentives, not established commercial outcomes. Apps may also prefer exclusive assistant relationships, and their recommendations may reflect business goals that differ from the user's interests.

An independent team should connect a third application using the published contract without changes to the agent's core. Record adapter code, integration time, app-specific branches, and the handling of context the schema cannot represent. Test replayed messages, disconnects, and a tab switch during a pending reply, measuring lost, duplicate, and misdirected outputs. If each app requires extensive bespoke orchestration, the proposed interoperability advantage is weak.

Agent-recommended app discovery and onboarding belong to this adoption problem and can follow the recurring collaboration. Evaluate the two presence designs separately from reasoning quality: vary app count and visit frequency, and measure visit-to-awareness latency, connection and request costs, stale observations, and recovery after disconnection. Include a previously authorized app outside the current task set, concurrent devices, and revoked publishing grants. This tests the queue's broader coverage and operating costs without assuming that a last-arriving event identifies the user's current app.

## **7. Conclusion**

Applications should accommodate the user's continuing relationship with a personal agent while retaining domain intelligence and authority over their services. Our position connects that relationship to application work through message delivery and context exchange, with user control over participation and delegation. The proposed architecture supplies an implementation path using established mechanisms. Its value depends on whether this arrangement improves continuity and collaboration at an acceptable cost, which the research agenda makes testable.

## **Use of generative AI**

Generative AI tools assisted with literature discovery, drafting, and editing of this manuscript. The authors remain responsible for the manuscript and its references.

## References

1. Zhenyu Zhao and Roy Zhao. 2026. *Runtime-Independent Persistent Agents: Preserving Identity, Memory, and Code Across Models, Harnesses, and Servers*. arXiv:2609.00546. [Paper](#).
2. Eric Horvitz. 1999. *Principles of Mixed-Initiative User Interfaces*. Proceedings of CHI '99. ACM. [Author's institutional publication page](#).
3. AG-UI contributors. *AG-UI Overview: The Agent-User Interaction Protocol*. Technical documentation. Accessed September 6, 2026. [Documentation](#).
4. CopilotKit. *Architecture and Agent Context*. Technical documentation. Accessed September 6, 2026. [Architecture](#); [Agent Context](#).
5. Goga Koreli. 2026. *Bring Your Own AI Agent Everywhere*. Public engineering note, August 20. [Design note](#).
6. Zhiyuan Wang, Erzhen Hu, Mark Rucker, and Laura E. Barnes. 2026. *PSI: Shared State as the Missing Layer for Coherent AI-Generated Instruments in Personal AI Agents*. arXiv:2604.08529. [Paper](#).
7. Stanford Digital Economy Lab. 2026. *The Future of Personal AI: Portable and Persistent Personal Memory through a Unified Human Context Protocol*. Report overview, March 16. [Report overview](#).
8. Roy Zhao and Zhenyu Zhao. 2026. *Code Is the Body: Agent-Owned Software Bodies for Recursive Evolution and Descent*. arXiv:2607.28691. [Paper](#).
9. Telegram. *Telegram Bot API*. Technical documentation, including `getUpdates` and `sendMessage`. Accessed September 6, 2026. [Documentation](#).
10. WHATWG. *HTML Living Standard: Server-Sent Events*. Accessed September 6, 2026. [Specification](#).
11. CloudEvents contributors. *CloudEvents Specification*, version 1.0.2. [Specification](#).
12. Anthropic. *Computer use tool*. Technical documentation, including limitations and token costs. Accessed September 6, 2026. [Documentation](#).